JAVA WEB PROJECT · GUVI

MarketHub

A multi-vendor e-commerce marketplace with Admin, Seller and Buyer roles, built with a React frontend and a Java Servlet + JDBC backend.

Aditya · github.com/aditya-glitch-bit/market-hub-allinone

PROBLEM UNDERSTANDING

Why MarketHub

Small sellers need one place to list products; buyers need one place to compare and order; admins need oversight.

Buyer

- Search, filter and sort products
- Cart and wishlist
- Place and track orders

Seller

- Create, edit, delete listings
- Inventory and low-stock alerts
- Sales reports and order processing

Admin

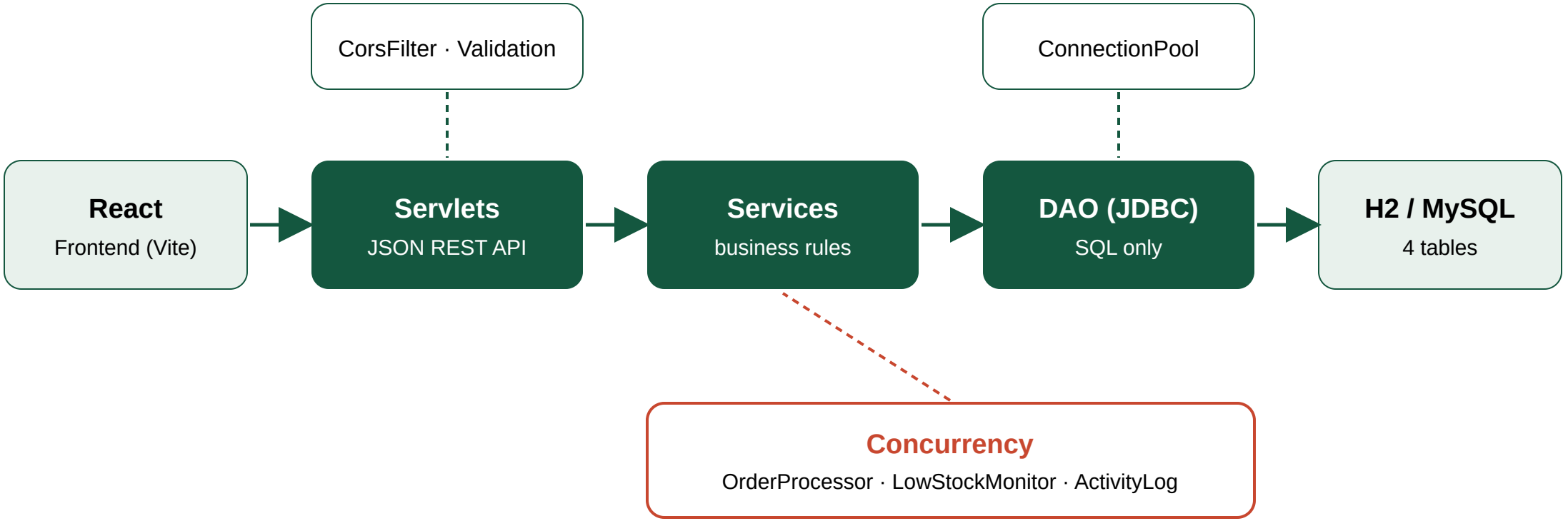
- Manage users and products
- Monitor orders
- Live activity feed

Key challenge Many buyers checking out at once must never oversell stock, and orders must be saved consistently.

SOLUTION DESIGN

Layered architecture

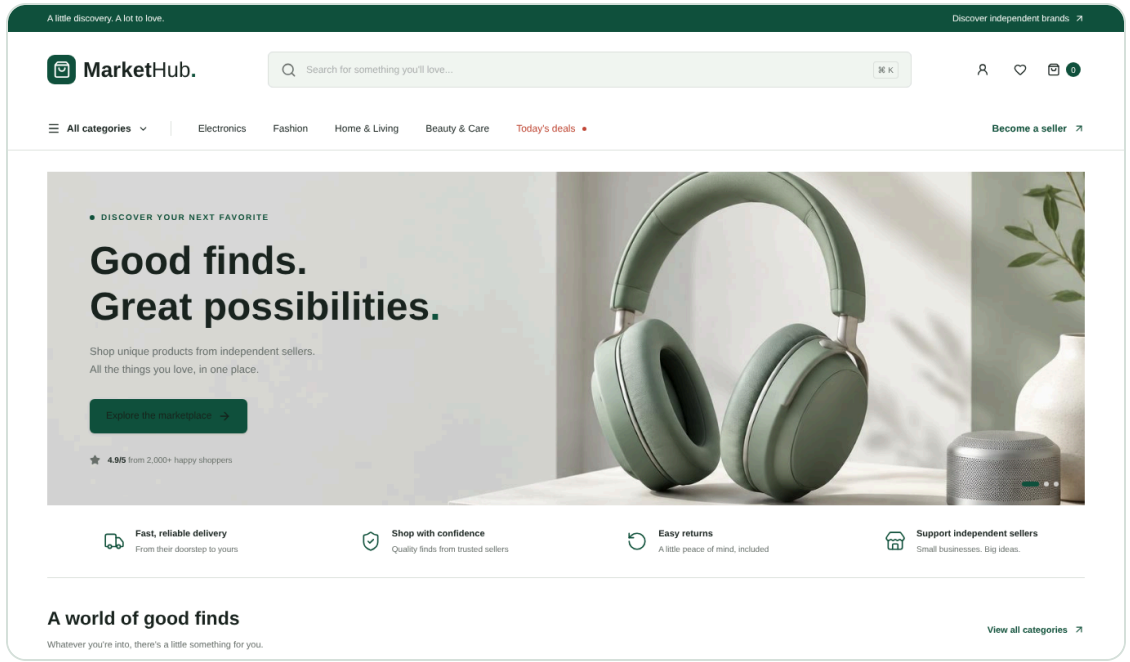
Each layer has one job, so the code stays modular and testable.



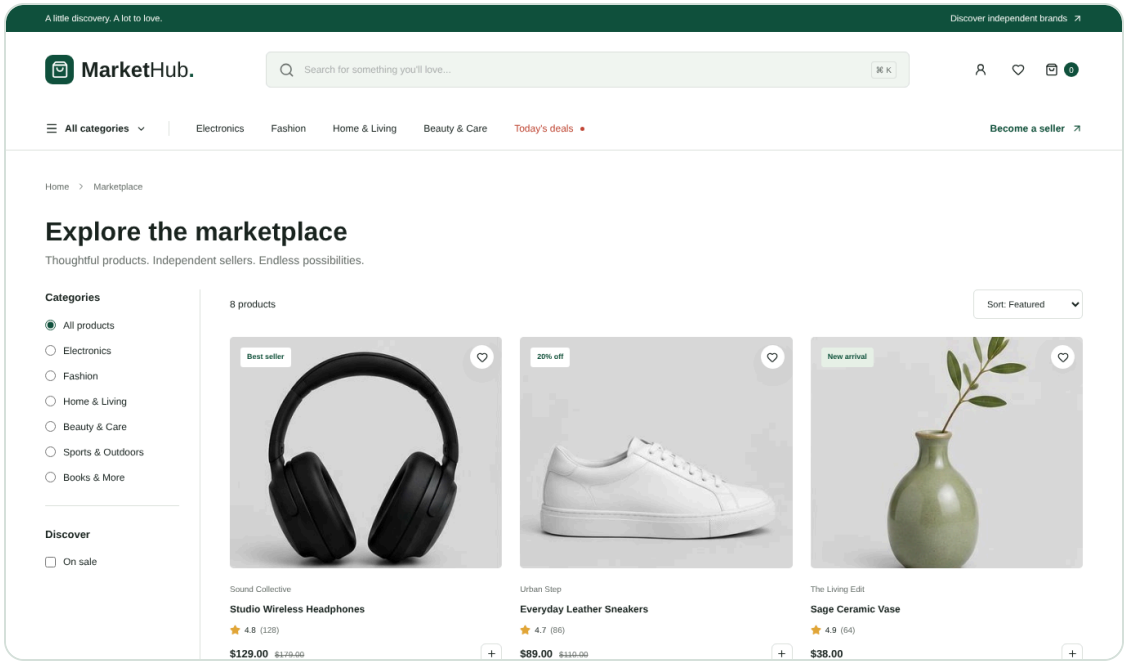
WORKING APPLICATION

Storefront

Responsive React interface with search, categories, product cards, wishlist and bag.



Homepage

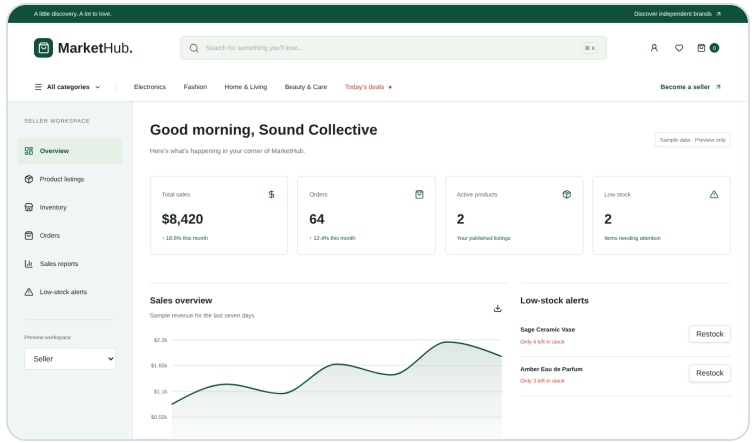


Catalog with filters and sorting

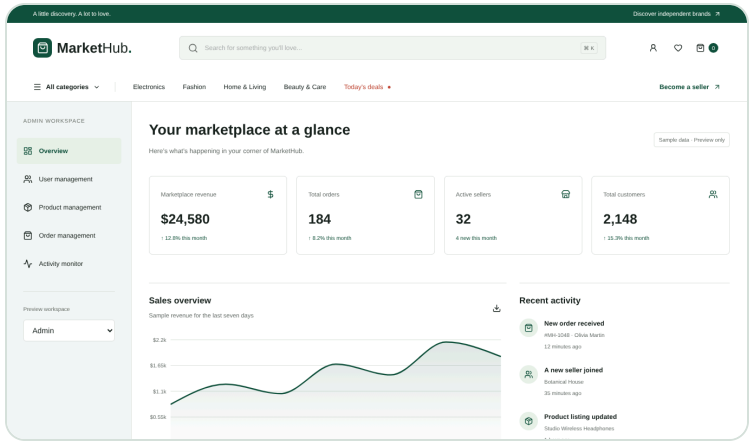
WORKING APPLICATION

Role dashboards

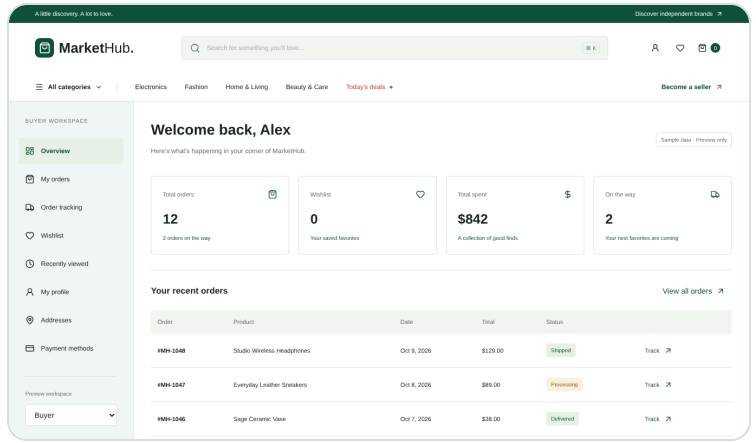
Each role gets its own workspace, fed by `/api/dashboard/{role}`.



Seller: sales, inventory, low-stock alerts



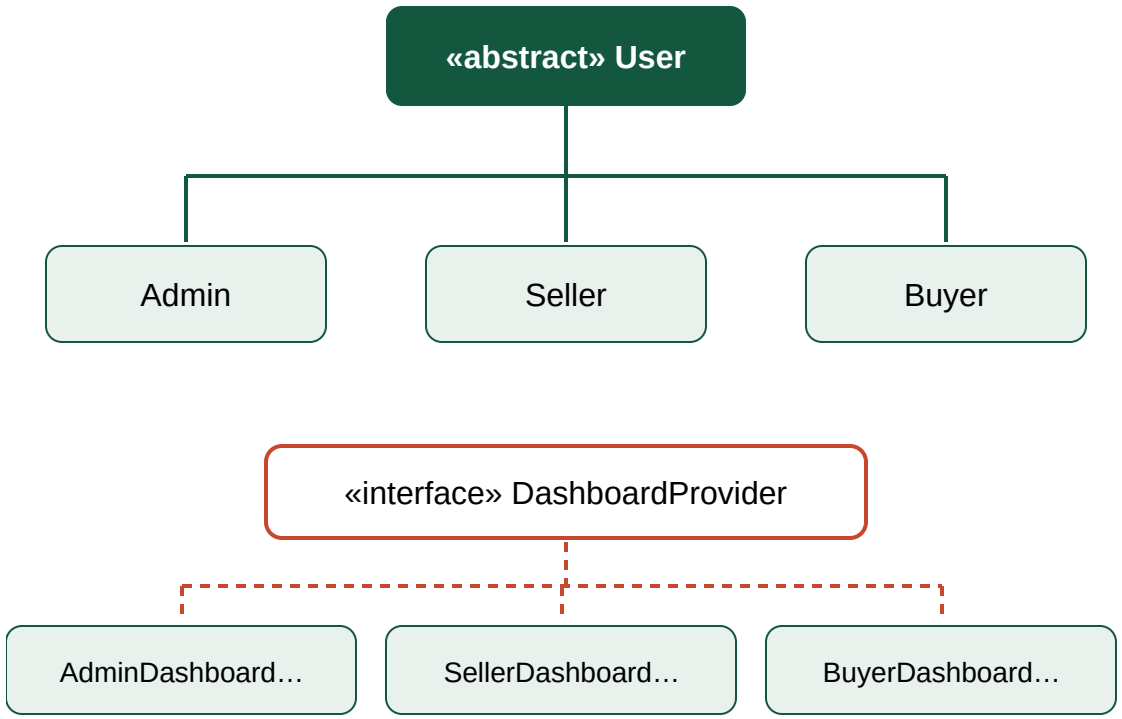
Admin: users, orders, activity



Buyer: orders, wishlist, profile

CORE JAVA · OOP

Inheritance, polymorphism, interfaces, exceptions



- **Inheritance:** `Admin`, `Seller`, `Buyer` extend abstract `User`; `UserFactory` builds them.
- **Polymorphism:** `DashboardService` picks a `DashboardProvider` by role at run time.
- **Interfaces:** `Identifiable<ID>`, `Discountable`, `Comparable`, DAO interfaces.
- **Exceptions:** checked hierarchy under `MarketHubException` (NotFound, Validation, OutOfStock, …) mapped to HTTP status codes in one place.

CORE JAVA · COLLECTIONS & GENERICS

Type-safe, reusable building blocks

GenericDao<T, ID>

One CRUD contract for every entity; bounded type `T extends Identifiable<ID>`. `AbstractJdbcDao` implements it once for all tables.

Collect (utility)

`groupBy`, `sumBy`, `countBy`, `topN` with PECS wildcards. They build the dashboard charts from `List`, `Map` and `SortedMap`.

LruCache<K,V> · Page<T>

Thread-safe cache on `LinkedHashMap` (access order). `Page<T>` wraps any paginated result.

Also: `Cart` keeps items in a map, `Product implements Comparable<Product>`, and `BigDecimal` is used for all money values.

CORE JAVA · MULTITHREADING & SYNCHRONIZATION

Safe under concurrent traffic

OrderProcessor

Producer/consumer. Request threads submit new orders; a worker pool takes them from a `DelayQueue` and ships them after a delay. Status change is a compare-and-set, so a manual update is never overwritten.

LowStockMonitor

A scheduled executor scans for low stock and publishes an immutable snapshot that dashboards read without locks.

InventoryService

Reserves stock with `UPDATE … WHERE stock >= ?`, taking all lines or none, so two buyers can never buy the last unit.

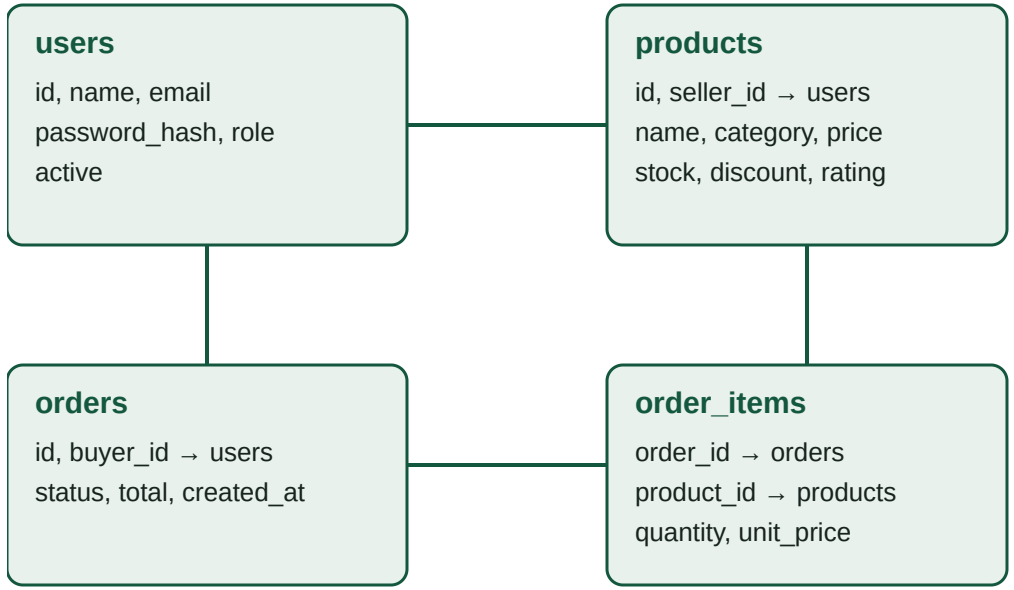
Also: `ActivityLog` (concurrent queue for the admin feed) and `ConnectionPool` (blocking pool shared by servlet threads).

DATABASE INTEGRATION · JDBC

DAO classes and transactions

All SQL is isolated in the DAO layer, using `PreparedStatement`.

- `JdbcProductDao`, `JdbcUserDao`, `JdbcOrderDao` extend `AbstractJdbcDao`
- `Database` + `ConnectionPool` manage connections
- `TransactionManager.inTransaction(...)`: stock reservation and order insert commit or roll back together
- H2 by default; MySQL by changing `MARKETHUB_DB_URL`
- Passwords stored with PBKDF2 and a random salt



SERVLETS & WEB INTEGRATION

REST API on Jakarta Servlets

Embedded Jetty serves the API; the React app calls it over JSON, allowed by `CorsFilter`.

Servlet / path	Responsibility
`AuthServlet` /api/auth/*	register, login, logout
`ProductServlet` /api/products/*	search, filter, sort, paginate; seller CRUD
`CartServlet` /api/cart/*	per-user cart
`OrderServlet` /api/orders/*	place orders, history, status updates
`DashboardServlet` /api/dashboard/*	admin / seller / buyer data
`UserServlet` /api/users/*	admin user management
`HealthServlet` /api/health	liveness check

All servlets extend `ApiServlet`, which handles JSON and turns each exception type into the right HTTP status.

RUBRIC COVERAGE

Where each criterion is met

Criterion	Evidence in the repository
OOP (polymorphism, inheritance, exceptions, interfaces)	`model/`, `service/dashboard/`, `exception/`
Collections & generics	`util/Collect`, `LruCache`, `Page`, `dao/GenericDao`
Multithreading & synchronization	`concurrent/`, `db/ConnectionPool`, `InventoryService`
Database operation classes	`dao/` and `dao/jdbc/`
JDBC connectivity	`db/Database`, `config/AppConfig`, `schema.sql`
Problem understanding & design	Three roles, layered architecture, 4-table schema
Servlets & web integration	`servlet/`, `MarketHubApplication` (Jetty)

SETUP

Run it

Backend (JDK 17, Maven)

```
cd backend
mvn package
java -jar target/markethub-backend.jar
# http://localhost:8080/api/health
```

Frontend (Bun or npm)

```
bun install
bun run dev
# http://localhost:5173
```

github.com/aditya-glitch-bit/market-hub-allinone

Thank you